TECHNICAL OVERVIEW · V1.0

IHA Platform

A medical tourism platform connecting international patients with hospitals, doctors and visa lawyers, from first enquiry to completed treatment.

5
Next.js portals, one per user type

6
Shared internal packages

137
SQL migrations in the schema

1
Supabase backend shared by all apps

CONTENTS

01 Executive summary

02 Architecture

03 Technology stack

04 Case lifecycle

05 Security model

06 Local development

07 Known issues and next steps

Prepared by
Sagar Nikam

Date
1 October 2026

Repository
IHA-Web-main

Classification
Internal

01

Executive summary

IHA runs the full journey of a medical tourism case: the patient's enquiry, doctor consultations, a treatment plan, payment, the visa, and the treatment itself. Each party works in its own portal, and all portals share one database.

Patients
Open a case, consult doctors, confirm and pay for a treatment plan, and manage visa and travel.

Doctors
Accept assigned cases, hold paid video consultations and submit treatment plans.

Hospitals
Manage their doctors and patients, and issue documents for confirmed treatment plans.

Visa lawyers
Collect documents, file the visa application and record its outcome.

Doctors, hospitals and lawyers register, pay a fee and wait for approval before they can work on cases. Platform admins handle that approval in an admin portal, which is still being built.

02

Architecture

One pnpm + Turborepo monorepo holds five Next.js 15 apps. They share six internal packages and one Supabase project for data, auth, files and realtime.

APPLICATIONS

App	Port	Used by	Main sections
apps/patient	`3000`	Patients	Cases, treatments, documents, payments, visa and travel, messages, profile
apps/hospital	`3001`	Hospitals	Case management, doctor management, patient management, documents
apps/visa	`3002`	Visa lawyers	Visa cases, documents, messages
apps/admin	`3003`	Platform admins	Dashboard only In progress
apps/doctor	`3004`	Doctors	Cases, consultations, messages

HOW THE PARTS CONNECT

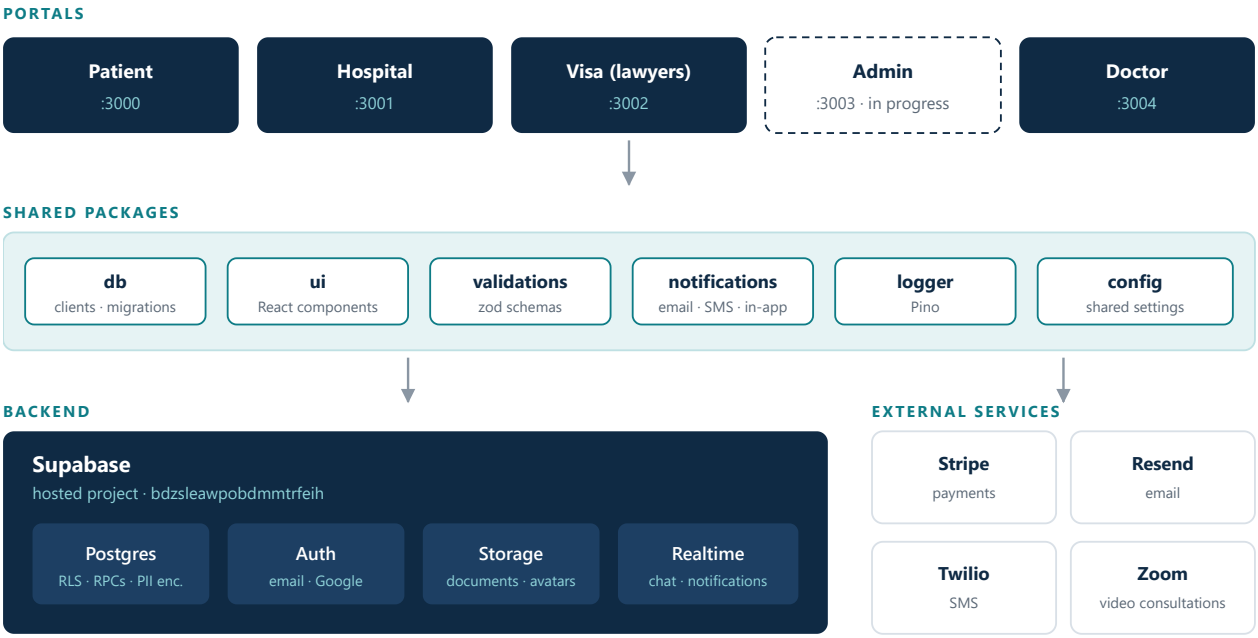


Figure 1. Every portal uses the same shared packages, which talk to one Supabase project and four external services.

03

Technology stack

Everything is TypeScript: Next.js apps on the front end and Supabase on the back end, organised as one monorepo.

Layer	Technology
Monorepo	pnpm 9 workspaces, Turborepo, TypeScript 5.8, Node 20 or later
Framework	Next.js 15 (App Router, Server Actions), React 19
Backend and database	Supabase: Postgres with row-level security, Auth, Storage, Realtime; `@supabase/supabase-js` and `@supabase/ssr`
Data fetching	TanStack Query 5; `nuqs` for state in the URL query string
UI	Tailwind CSS 3, Radix UI primitives (shadcn style, with `class-variance-authority` and `tailwind-merge`), Lucide icons, Sonner toasts, Embla carousel
Forms and validation	React Hook Form and Zod (shared in `packages/validations`); `libphonenumber-js` for phone numbers
Translations	`next-intl`, with message files such as `apps/patient/messages/en.json`

Layer	Technology
Payments	Stripe (server SDK and `@stripe/react-stripe-js` in the browser)
Email	Resend with React Email templates
SMS	Twilio
Video	Zoom, for consultations; probably called through its API directly, since there is no Zoom npm package
PDFs	`pdf-lib` for invoices, receipts and visa documents
Logging	Pino (`packages/logger`)
Tooling	ESLint 9, Prettier, Supabase CLI for migrations and generated types

04

Case lifecycle

A case moves through six steps and changes hands from the patient to doctors, then to the hospital and a lawyer.

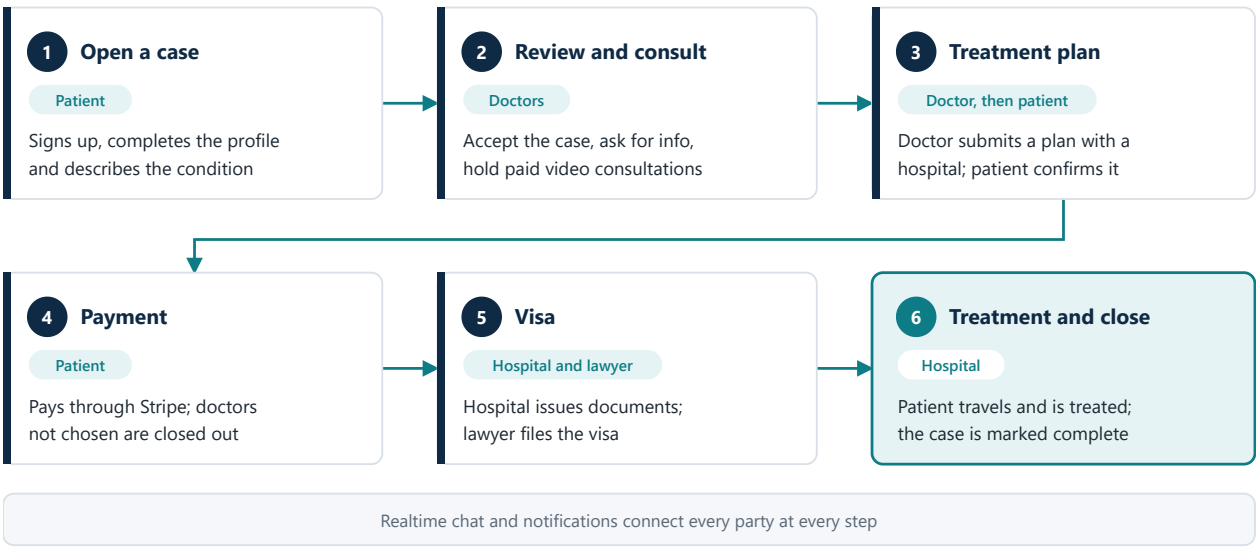


Figure 2. The six steps of a case and who acts at each one.

05

Security model

Most security lives in the database, not the apps, so all five portals follow the same rules.

Row-level security

Policies decide who can see each case. For example, a hospital only sees a case once its treatment plan is confirmed.

Encrypted personal data

Names, demographics, case descriptions, visa forms and provider credentials are encrypted in Postgres with `encrypt_pii`.

Writes through RPCs

Most writes go through `security definer` functions rather than direct table inserts.

Signup trigger

`handle_new_patient_user` creates a `patients` row for every new user without a non-patient role.

Account deletion

Patient accounts are soft-deleted first, then hard-deleted by a scheduled job. Audit logs record each step.

06

Local development

This machine has Node 24 but no pnpm or Docker, so each app runs from its own Next.js binary against the hosted Supabase project.

```
# Patient portal (use the port from the Applications table for other apps)
cd apps/patient
./node_modules/.bin/next.cmd dev --port 3000
```

- Supabase project: `bdzsleawpobdmmtrfeih` (hosted; no local database).
- Each app needs a `.env.local` with the Supabase URL and keys. `apps/patient/.env.local` has them.
- The Resend, Twilio, Stripe and Zoom keys are empty, so email, SMS, payments and video calls won't work yet.
- The patient app opens at `http://localhost:3000` and redirects to `/login`.

07

Known issues and next steps

The hosted database looks only partly migrated, and that blocks the patient portal at onboarding.

ISSUE · MISSING PATIENTS ROW

Signing in with Google leads to `/profile/setup`, which fails with `No patients row for user 440cd87a-…`. The account has no role, so the signup trigger should have created the row.

LIKELY CAUSE · NOT CONFIRMED

The account was created at 01:52 UTC on 23 September 2026, probably before the trigger existed in the hosted project. A sign of partial migration is that the `lawyers` table is missing, even though the repo has a migration for it.

NEXT STEPS

1. **Apply the missing migrations** to `bdzsleawpobdmmtrfeih`, with `supabase db push` or by running the SQL files in order.
2. **Repair the account:** delete the user and sign in again, or insert the `patients` row with `encrypt_pii`.
3. **Add the service keys** for Resend, Twilio, Stripe and Zoom to `.env.local`.